BUSINESS ANALYTICS · PRESENTATION STUDY GUIDE

skillet

What every number on the app means, the maths in plain language, and how to present it

A cheat sheet, a guide to the web app, the modelling explained, a timed presentation plan with a demo script, likely questions with answers, and test scenarios.

COMPANION	PROJECT_REPORT.pdf (full report with derivations)
DATE	October 2026

How to use this guide

This guide is for learning the project well enough to present it and answer questions.

- **Part 1** is a cheat sheet: memorise it.
- **Part 2** goes through every element on the web app and says what each number means.
- **Part 3** explains every mathematical model step by step: the question, the idea, each term of the formula and how to read the result.
- **Part 4** is a minute-by-minute presentation plan with a live-demo script.
- **Part 5** has the questions you are likely to be asked, with answers.
- **Part 6** lists test scenarios with the exact numbers the app should show, so you can rehearse the demo and prove the app matches the model.

The running example throughout is scenario **B1**:

- **Profile:** a full-stack developer in the United States with 5 years of experience who uses AI tools.
- **Stack:** Python, JavaScript, SQL, PostgreSQL and Docker.

To start the app:

```
.venv/bin/python -m streamlit run app/streamlit_app.py
```

Part 1: Cheat sheet

One sentence. skillet tells a developer which five skills to learn next, using what similar developers know and want, and how much each skill is associated with in pay, from the 2025 Stack Overflow Developer Survey.

The data

- 19,210 working professional developers, 122 skills, 156 countries, pay in US dollars.
- 18,388 developers in the salary models (USD pay known, extreme outliers removed).

The three building blocks

Block	Method	Question it answers
Salary evidence	Regression of log pay on each skill, holding experience and country fixed	How much more do people with this skill earn than similar people without it?
Personalisation	K-nearest neighbours (Jaccard similarity), weighted by similarity	What do developers like me know and want to learn?
Plan value	One joint regression on the five recommended skills	What are all five worth together, without double-counting?

Numbers to remember

Result	Value
Pay per extra year of experience	**+3.7%** (R^2 = 0.128)
Full model (experience + country + all skills)	R^2 = **0.559**
Highest-paying skills	Scala +35%, Snowflake +33%, Terraform +31%, ClickHouse +31%, Databricks SQL +30%
Lowest-paying skills	Microsoft Access −27%, Delphi −24%, VB.NET −22%, PHP −20%, Laravel −19%
AI users' pay, same country and experience	**+9.9%** (95% CI +7.3% to +12.7%). The raw comparison points the *other* way (Simpson's paradox).
Job satisfaction from skills	Hard to predict: best AUC = 0.57
Personas	3: .NET stack (17.9%), Lean generalist (46.2%), Modern JS / web (35.9%)
Recommender accuracy	Finds a hidden skill in its top five **78%** of the time, against 72% for "most popular" and 5% for "highest salary"

The scoring formula, in words:

> *score = relevance (share of peers who know it) + 0.5 × value (salary coefficient) + 0.3 × demand (share of peers who want it) + small bonuses for statistical significance and AI models.*

Three ideas to keep coming back to

1. **Country fixed effects.** Pay is compared within each country, so rich-country skills don't look valuable just because of where their users live.
2. **The joint total is smaller than the sum.** Skills come in bundles and share one premium; the joint regression doesn't double-count.
3. **Association, not causation.** "Scala users earn 35% more than comparable developers" is not the same as "learning Scala will raise your pay by 35%".

Part 2: Every element on the web app

2.1 Left rail: your inputs

Element	What it is
"2025 Stack Overflow survey · 19,210 developers"	The dataset: working professional developers left after cleaning.
Role	Your job type (32 survey roles). Defaults to the most common, Developer Full Stack. It **filters** the peer group.
Years of professional experience	0–50. Peers must be within **±10 years** of this.
Country	Peers must be from the same country, and pay is compared within country.
"I use AI tools in my workflow"	Peers must match your AI-tool use.
Your current stack (six boxes)	The skills you have now, up to 6 per group. They find your most similar peers, decide your persona, and are never recommended back to you.
"121 skills scored · salaries in USD"	121 of the 122 skills have enough users (at least 50) for a salary estimate.

Key point. The profile inputs (role, experience, country, AI) decide **who you're compared with.** The stack decides **who is most similar to you** within that group.

Where some skills are listed:

- **Cloud platforms:** Vite, pnpm, npm, Yarn, Webpack, NuGet, Docker. This is how the survey groups them.
- **Dev environments:** Visual Studio.
- **Web frameworks:** React, Node.js, Express, Next.js, ASP.NET, ASP.NET Core, Laravel.

2.2 The header line

> *"Your 300 nearest peers in skill space, drawn from 731 developers who share your profile (developer full stack · United States · 0–15 years · AI users)."*

- **731:** everyone in the survey who matches all four profile filters.
- **300:** the 300 of them whose stacks are most similar to yours. If fewer than 300 match, all of them are used (in India, 277).
- **Relaxation note** (rare profiles only): if fewer than 50 developers match, filters are dropped one at a time (AI use first, then experience, then country, then role), and the page says which were dropped.
- **Empty stack ("cold start"):** the line says the page is showing the highest-value skills in your group, because there's no stack to compare yet.

2.3 The sky chart ("Plate I. Skill space, 2025")

Feature	Meaning
Each dot	One of the 122 skills.
Position	Skills that developers use **together** sit **close together**. The layout comes from t-SNE on co-usage (1 − Jaccard similarity), so the JavaScript tools cluster together, the .NET tools together, and so on.
Axes	Meaningless, so they are hidden. Only "near" or "far" matters.
Dot size	The skill's salary effect (legend: +5%, +15%, +30%).
White joined stars	Your stack, drawn as your "constellation".
Amber rings	The five recommendations.
Hover	Skill name, salary effect and p-value.

What to say about it. The recommendations sit next to your constellation because they are what people with stacks like yours also use.

2.4 The plan panel ("All five, estimated together")

Number (B1)	Meaning
+24% (big number)	Learning all five is associated with about 24% higher pay, at the same experience and country. It is computed from **one regression with all five skills together**, so overlap isn't double-counted.
"95% interval +20% → +28%"	The true combined effect is very likely between +20% and +28%. A narrow interval means a precise estimate.
"3 of 5 significant"	In that joint regression, 3 of the 5 skills have $p < 0.05$. "Significant" means "clearly not zero", not "large". A skill can even be significantly negative jointly: npm is −3.2%.

Each of the five rows (example: "TypeScript · 67% of your peers know it · p = … · +6.7%"):

- **Rank (1–5):** the order of the recommendation scores.
- **"67% of your peers know it":** among your 300 nearest peers, weighted so the most similar count more, 67% use TypeScript.
- **p-value:** how sure we are that TypeScript's salary effect is not zero. Smaller is more certain; 0.05 is the usual cut-off.
- **+6.7%:** TypeScript's **stand-alone** salary effect. Developers who know it earn about 6.7% more than those who don't, at the same experience and in the same country.
- **Italic tag** ("up 2", "down 1", "new"): how the skill moved since your last change, so the effect of every input is visible.

The three facts at the bottom:

Fact (B1)	Meaning
Median pay, your group: $130,000	Median annual pay of your nearest peers. With an empty stack, it's the whole matching group.

Fact (B1)	Meaning
Premium vs same country and experience: −2.0%	How much more or less your peers earn than the **average** developer with the same experience in the same country. It is the average residual of `log pay ~ experience + country`. It is context only: it would be the same for every candidate skill, so it can't change the ranking.
Persona (from your stack): Lean generalist stacks	Which of the three developer types your stack belongs to (K-means). It depends only on skills, not on role, country or experience.

Why +24% and not the sum. The five stand-alone effects add to about +41%, but the joint model gives +24%. Skills that come as a bundle share one premium. For example, Docker and Kubernetes users overlap so much that each one's stand-alone number already contains much of the other's premium.

2.5 "The five, in detail" (evidence table)

Column	Meaning
#	Rank.
Skill and evidence	Name, plus a sentence like "Weighted by similarity, 67% of your 300 nearest peers know it and 47% want to learn it."
Salary effect	Stand-alone % pay association, from that skill's own regression with experience and country held fixed.
95% interval	Likely range of that effect. If it doesn't include 0%, the effect is significant.
p-value	Probability of an effect this large if the true effect were zero. Below 0.05 counts as significant.
Survey users	How many developers in the salary sample know it. More users means a more reliable estimate.
Peers know	Similarity-weighted share of your nearest peers who use it. In cold start it reads "Profile matches know" and covers your whole group.
Peers want	Share of your nearest peers who said they **want to work with** it next year: a demand signal. It shows "—" in cold start.

2.6 "Who you are in the data" (persona)

Example: "Lean generalist stacks. The K-means model places your stack in this cluster of 8,868 developers (46.2% of the sample). Their stacks list 10 common skills on average..."

Persona	Share (developers)	Average stack size	Distinguishing skills
.NET stack	17.9% (3,439)	21	ASP.NET, ASP.NET Core, NuGet, C#, Visual Studio
Lean generalist stacks	46.2% (8,868)	10	Small stacks; C++, C, Swift, Java, Oracle
Modern JS / web tooling	35.9% (6,903)	27	pnpm, Express, Yarn, Webpack, Vite

The tags are the skills with the highest **lift**: how many times more common the skill is inside the cluster than in the whole survey.

2.7 "Skill ROI landscape" chart

- **What it shows:** the 20 skills with the **highest salary effect in the whole survey**. Each star is the estimate and each line its 95% interval.
- **Brightness** means significance: brighter is a smaller p-value. The scale runs from $p \approx 1$ to $\leq 10^{-100}$.
- **Amber ring:** the skill is also in your plan.
- **It does not change with your inputs.** It is the market-wide view, and your plan is the personalised view.

2.8 "What if I also learned these?" calculator

- **What it does:** pick up to 8 skills (not ones you already have, and not the five picks) to see their **combined** effect from one joint regression.
- **What it shows:** the big percentage, its 95% interval, "k of n coefficients clearing $p < 0.05$", and a chart of each skill's **joint** contribution with its interval (green positive, red negative).
- **Example:** Kubernetes + Terraform + Snowflake comes to **+73.7%**, even though separately they add to +92%.

2.9 "How these recommendations are scored" (expandable notes)

This section repeats, in plain language, the scoring formula, the peer rules, the method pipeline, the data source and the limitations.

Part 3: The mathematical modelling, explained

This part explains every model in the project. For each one: what question it answers, the idea behind it, what each part of the formula means, and how to read the result.

3.0 The big picture

The project answers one question: *which five skills should this developer learn next?* A good answer needs three things:

1. **Value.** Is the skill associated with higher pay? This comes from regression models.
2. **Fit.** Do people like this developer actually use it? This comes from nearest-neighbour matching.
3. **Demand.** Do people like this developer want to learn it? This comes from the survey's "want to work with" question.

Everything else in the project (tests, classifiers, clustering) either supports these three or describes the population.

3.1 Measuring pay: why we use the logarithm

The problem. Raw salaries are lopsided: most developers earn moderate amounts and a few earn huge amounts. They also differ enormously by country (median $130,000 in the US, $15,000 in India). If we modelled raw dollars, a skill would be "worth" a fixed amount, say +$10,000. That is a small raise in the US and several times the median pay in India.

The idea. Model the **logarithm** of pay instead: $y = \log(\text{pay})$. Logs turn multiplying into adding. "Earns 10% more" becomes "log pay is about 0.1 higher", whatever the starting salary. So every effect the model estimates is a **percentage**, which means the same thing in every country.

Reading a coefficient. If a skill has coefficient β, people with the skill earn e^{β} times as much:

$$\%\ \text{effect} = (e^{\beta} - 1) \times 100$$

For example, TypeScript has $\beta = 0.065$, so $e^{0.065} = 1.067$, which is **+6.7% pay**. For small β the percentage is roughly 100β; for large ones (Scala, $0.30 \rightarrow +35\%$) the exact formula matters.

3.2 Simple regression: experience and pay

Question. How much does each year of experience add to pay?

$$\log(\text{pay}) = \beta_0 + \beta_1 \times \text{experience}$$

- β_0 **(intercept):** predicted log pay at zero experience.
- β_1 **(slope):** how much log pay rises per year.
- **How it's estimated:** OLS (ordinary least squares) draws the line that makes the squared vertical gaps between the data and the line as small as possible.

Result. $\beta_1 = 0.036$, so each year of experience is associated with **+3.7% pay**.

$R^2 = 0.128$. R^2 is the share of pay differences the model explains. Experience alone explains about 13%; the other 87% comes from country, skills, employer, negotiation and luck.

3.3 Multiple regression with country fixed effects

Question. If we put experience, every skill and every country in one model, how much of pay can we explain?

$$\log(\text{pay}) = \beta_0 + \beta_x \,\text{experience} + \sum_j \gamma_j \,\text{skill}_j + \sum_c \alpha_c \,\text{country}_c$$

The new ingredient: country fixed effects. Each country gets its own yes/no variable (α_c), with the US as the baseline. Each α_c soaks up that country's average pay level. With these terms present, every other coefficient is effectively estimated by comparing **developers within the same country**.

Why this matters. Snowflake is used mostly in the US, where pay is high. Without country terms, Snowflake would look valuable partly *because its users live in the US*. With them, Snowflake users in Germany are compared with non-users in Germany, US users with US non-users, and so on.

Result. $R^2 = 0.559$: experience, country and skills together explain 56% of pay differences. Adjusted R^2 (0.553) penalises the model for its 274 parameters and is barely lower, so the model is not just overfitting.

Multicollinearity (VIF). When predictors are highly correlated, their coefficients become unstable. VIF measures this: 1 means no problem, and above 5–10 is a concern. Ours are 1.0–1.6, so the estimates are stable.

3.4 Per-skill salary ROI: the value of each skill

Question. For each skill: how much more do people who know it earn than comparable people who don't?

The model. One separate regression per skill:

$$\log(\text{pay}) = \beta_0 + \beta_x \,\text{experience} + \beta_j \,(\text{knows skill } j) + \text{country effects}$$

Reading β_j. This is the pay difference between developers *with* skill j and *without* it, at the **same experience** and in the **same country**. That is exactly what a learner wants to know. Converted with $(e^{\beta} - 1) \times 100$, it is the skill's "salary ROI".

Why one regression per skill, not one big model? In the big model of §3.3, each coefficient means "the effect of j with every other skill held fixed". Skills come in bundles (Docker with Kubernetes, React with TypeScript), so in that model they split credit in hard-to-interpret ways. The per-skill model gives one clear, comparable number per skill, which is right for **ranking**. For **adding** skills together we use a different model (§3.5).

How sure are we? Each estimate comes with:

- **Standard error (SE):** how much the estimate would wobble with a different sample.
- $t = \beta/\text{SE}$**:** how many standard errors the estimate is from zero.

- **p-value:** if the skill truly had zero effect, the chance of seeing an estimate this far from zero. Below 0.05 we call it significant.
- **95% interval:** $\beta \pm$ about 2 SE, converted to percent: the range of effects consistent with the data.

Results. Scala +35%, Snowflake +33% and Terraform +31% at the top; Microsoft Access −27%, Delphi −24% and PHP −20% at the bottom. 98 of the 121 skills are significant.

3.5 The joint uplift: what five skills are worth together

The trap. It is tempting to add the five per-skill effects, but that double-counts. Almost everyone who knows Kubernetes also knows Docker, so Docker's stand-alone +16% already includes part of the Kubernetes premium, and vice versa. Adding them counts the shared part twice.

The fix. Put all five skills into **one** regression:

$$\log(\text{pay}) = \beta_0 + \beta_x\,\text{experience} + \beta_1\,\text{skill}_1 + \cdots + \beta_5\,\text{skill}_5 + \text{country effects}$$

Each coefficient now measures what that skill adds **on top of the other four**, so the overlap is counted once.

The total. $\theta = \beta_1 + \beta_2 + \cdots + \beta_5$, converted to a percentage.

Its uncertainty. The variance of a sum is not just the sum of the variances. Coefficients of overlapping skills move together: if one is estimated high, the other tends to be estimated low. So

$$\text{Var}(\hat{\theta}) = \sum_j \text{Var}(\hat{\beta}_j) + \sum_{j \neq k} \text{Cov}(\hat{\beta}_j, \hat{\beta}_k)$$

This gives a correct 95% interval for the total. Adding up the separate intervals' endpoints would not.

Example. For the worked example (TypeScript, VS Code, npm, React, AWS):

- The stand-alone effects add up to about **+41%**.
- The joint model gives **+24%** (95% CI +20% to +28%).
- In the joint model AWS keeps its +21%, while VS Code and React drop to about 0. They are part of the same bundle, not separate sources of pay.

This is the big number on the app's plan panel.

3.6 Hypothesis tests: checking beliefs

All tests ask whether a difference in the data is bigger than random chance would produce. The **p-value** answers that: below 0.05, we reject the "no difference" hypothesis (H_0).

Welch's t-test: do AI users earn differently?

$$t = \frac{\text{mean}_1 - \text{mean}_2}{\text{standard error of the difference}}$$

The t statistic is the difference in averages divided by how noisy that difference is. Welch's version doesn't assume both groups have the same spread.

- **Result:** $t = -6.15$. AI users appear to earn **less**.

The same question, controlled. We regress log pay on AI use plus experience and country fixed effects.

- **Result:** AI users earn **+9.9% more** (CI +7.3% to +12.7%).

Why the sign flips (Simpson's paradox). AI tools are used more in lower-paying countries. Pooling everyone together, AI users look poorer because of *where they live*. Within the same country and at equal experience, they earn more. Lesson: always compare like with like.

Chi-square test: are two categories related?

- **How it works:** build a table, for example AI use by remote-work type. For each cell, compare the observed count with the count **expected** if the two were unrelated (row total × column total ÷ grand total). χ^2 adds up the squared gaps, scaled by the expected counts. A big χ^2 means the variables are related.
- **Cramér's V:** turns χ^2 into a strength between 0 and 1, because χ^2 grows with sample size.

Results:

- AI use × remote work: p = 0.06, V = 0.02. No real relationship.
- Role × JavaScript: V = 0.355. Strongly related (front-end and full-stack developers use JavaScript far more).

3.7 Classification: can we predict job satisfaction?

Question. From skills, experience, AI use, remote work and company size, can we predict who is highly satisfied (8 or more out of 10)?

Logistic regression. A linear score is passed through an S-shaped curve so the output is a probability between 0 and 1:

$$P(\text{satisfied}) = \frac{1}{1 + e^{-(w_1 x_1 + w_2 x_2 + \cdots + b)}}$$

- **Positive weight:** that feature raises the probability.
- **Fitting:** the weights are chosen to make the observed outcomes as likely as possible (minimising log-loss).
- **Class weights:** keep the model from favouring the bigger class.
- **Penalty:** stops the weights from growing too large (overfitting).

Decision tree. It asks yes/no questions ("experience > 10?", "works remotely?") and splits people into increasingly pure groups, measured by Gini impurity. The depth is capped at 6 so it doesn't memorise the data.

Naive Bayes. It uses Bayes' rule, assuming each feature independently shifts the odds of being satisfied.

How we judge them.

- **Data split:** train on 75% of the data, test on the unseen 25%.
- **AUC:** the probability that the model ranks a random satisfied person above a random unsatisfied one. 0.5 is coin-flipping and 1.0 is perfect.

- **Threshold:** the cut-off maximises Youden's J = true-positive rate − false-positive rate.

Result. AUC ≈ **0.57** for all three models.

What to say. Satisfaction is barely predictable from these features. It depends on things the survey doesn't measure: manager, team, autonomy. This is a finding, not a failure, and three different methods agreeing makes it convincing.

3.8 Clustering: finding developer personas

Question. Are there natural *types* of developers, based on the skills they use?

Step 1, standardise. Take the 86 skills used by at least 5% of developers (rare skills are mostly noise) and rescale each column so no single skill dominates.

Step 2, PCA (compression). 86 dimensions is a lot. PCA finds new axes ("components") that capture the most variation. One component might be "JavaScript ecosystem versus not". We keep the first **51 components**, which retain 80% of the information.

Step 3, K-means (grouping). K-means places k centre points and assigns each developer to the nearest one. It then moves each centre to the middle of its group and repeats until nothing changes. The goal is to minimise the total squared distance from developers to their centre.

Choosing k: the silhouette. For each developer compare a (average distance to their own cluster) with b (average distance to the nearest other cluster):

$$\text{silhouette} = \frac{b - a}{\max(a, b)}$$

It ranges from −1 to 1; higher means better-separated clusters. The best k is **3** (silhouette 0.085). That is low, but normal for sparse yes/no data, and the groups are still clearly interpretable.

Naming the clusters: lift.

$$\text{lift} = \frac{\text{share of the cluster using the skill}}{\text{share of everyone using it}}$$

A lift of 4.8 means the skill is 4.8 times more common in that cluster than overall. The highest-lift skills name the cluster:

- **.NET stack** (18%): ASP.NET, C#, NuGet.
- **Lean generalist** (46%): small stacks of about 10 skills.
- **Modern JS / web tooling** (36%): pnpm, Vite, Yarn, Express.

Hierarchical clustering (Ward). Start with every developer as their own group and repeatedly merge the two groups whose merger adds the least spread. The resulting tree (dendrogram) shows the same structure visually.

3.9 Factor analysis: hidden "ecosystems"

Idea. PCA just compresses. Factor analysis assumes there are hidden causes (**factors**) that make certain skills appear together. For example, an underlying ".NET-ness" makes people use C#, ASP.NET and Visual Studio together.

$$\text{observed skills} = \text{loadings} \times \text{hidden factors} + \text{skill-specific noise}$$

Loadings say how strongly each skill is driven by each factor.

How many factors.

- **Kaiser rule:** keep factors with eigenvalue > 1, meaning each explains more than one skill's worth of variance. That gives 18.
- **Parallel analysis:** compares against random data and suggests 15.
- We keep the 18.

Varimax rotation. The factors are rotated so each skill loads strongly on one factor and weakly on the others, which makes them easy to name.

Result. The 18 factors correspond to technology ecosystems (JavaScript front-end, .NET, Java, OpenAI tools, Linux/DevOps, systems languages, mobile, ...) and explain 50% of variance. This independently confirms that skills cluster by ecosystem.

3.10 ANOVA: does pay differ by persona?

Idea. Split total variation in pay into two parts:

- **Between groups:** how far each persona's average is from the overall average.
- **Within groups:** how spread out people are inside each persona.

$$F = \frac{\text{between-group variation per group}}{\text{within-group variation per person}}$$

A large F means the group averages differ more than random spread would explain.

Result. $F = 16.9$, $p < 10^{-7}$, so the persona averages do differ; .NET developers earn the most.

But $\eta^2 = 0.002$. η^2 is the share of pay variation explained by persona: only 0.2%.

Lesson. A result can be *statistically significant* (clearly real) without being *practically important* (big). With 19,000 people, even tiny differences become significant.

Bonferroni. Comparing three pairs raises the chance of at least one false alarm. Multiplying each p-value by 3 corrects for that. All three pairs stay significant.

3.11 The skill network

Idea. Treat skills as nodes and connect two skills when they are often used together.

Jaccard similarity between skills j and k:

$$J = \frac{\text{people who use both}}{\text{people who use either}}$$

An edge is drawn when $J > 0.30$.

Centrality. A skill's degree centrality is the share of other skills it connects to. The hubs (npm, Docker, SQL) are the connective tissue of the ecosystem.

Modularity (Q). Q measures whether a grouping puts more links *inside* groups than chance would.

- The three personas give $Q = 0.21$.
- An algorithm that searches for the best grouping finds $Q = 0.38$.

So the real structure is ecosystems, consistent with factor analysis.

3.12 The recommender: how the five picks are chosen

This is the heart of the app. It works in five steps.

Step 1, find comparable developers. Keep everyone with your role and country, experience within ±10 years, and the same AI use. For the default profile that is **731** developers.

- If fewer than 50 match, the app drops the least important filter first (AI → experience → country → role) and tells you which it dropped.

Step 2, find the most similar stacks. Among those developers, measure similarity with **Jaccard**:

$$\text{similarity} = \frac{\text{skills you share}}{\text{skills either of you has}}$$

Why Jaccard? It ignores skills **neither** of you has. With 122 skills, two people both lack about 110 of them; counting those shared absences would make everyone look alike. The **300 most similar** developers become your "peers".

Step 3, weight by similarity. A peer who shares 80% of your stack counts more than one who shares 20%. The weights are each peer's similarity divided by the total, so they add to 1. This matters when the group is small: without weights, all 277 Indian developers would count equally whatever your stack.

Step 4, measure relevance and demand. For each skill you don't have:

- **know(j):** the weighted share of peers who use it. *Is it normal for people like me?*
- **want(j):** the weighted share who said they want to learn it. *Is it in demand among people like me?*

Step 5, score and rank.

$$\text{score}(j) = \text{know}(j) + 0.5\,\beta_j + 0.3\,\text{want}(j) + 0.05\,[\text{significant}] + 0.02\,[\text{AI model and you use AI}]$$

- **Relevance + value + demand + reliability.** The weights put them on comparable scales: shares run from 0 to 1, coefficients from about −0.3 to +0.3.
- **Eligibility:** the skill must have a **positive** pay effect (so PHP is never recommended) and be known by more than 2% of peers.

Worked example (TypeScript):

$$0.671 + 0.5 \times 0.065 + 0.3 \times 0.472 + 0.05 = 0.895$$

That's the highest score (67% of peers know it, 47% want it), so it ranks first.

Context numbers, not used in the score:

- **Median pay** of your peers.

- **Premium.** Predict everyone's pay from experience and country alone. Each person's **residual** (actual minus predicted) is how much more they earn than expected. Your peers' weighted average residual is the premium. In the example it is −2%: your peers earn slightly less than typical developers with the same experience and country.
- **Why it's not in the score:** it would add the same amount to every skill, so it can't change the order.

Empty stack ("cold start"). With no stack there are no neighbours, so the ranking uses salary coefficient + 0.3 × the share of your profile group who know the skill, plus the same bonuses.

Persona. Your stack goes through the same standardise → PCA → K-means pipeline as in §3.8, so it lands in the same cluster the analysis would assign.

3.13 The sky chart (t-SNE)

Goal. Draw 122 skills on a flat page so that skills used together sit together.

How it works.

1. Start from the distance between every pair of skills, $1 - J$.
2. t-SNE turns these distances into "how likely is skill k to be a neighbour of skill j" probabilities.
3. It arranges points on the page so the on-page neighbour probabilities match the real ones as closely as possible (it minimises the KL divergence between the two).

How to read it. Only *neighbourhoods* mean something. The axes and long-range distances don't, which is why the chart has no axis numbers.

3.14 Validation: does the recommender actually work?

Setup. Take 1,000 real developers the system wasn't tuned on and remove each from their own peer pool, so they can't "see themselves".

Test A, hidden skills. Hide two of each person's skills and ask for five recommendations from the rest. Did at least one hidden skill appear? (**hit@5**)

- **Our model: 78%.**
- Popularity baseline: 72%. Highest-salary baseline: 5%.

Test B, wanted skills. What share of the five picks did the person say they *want* to learn? (**want-precision@5**)

- **Ours: 15.8%**, against 12.6% (popularity) and 5.2% (highest salary).

Uncertainty. Bootstrap: resample the test developers 2,000 times and recompute; the middle 95% of results gives the interval.

Weights. We tried a grid of weights on one half of the data and checked them on the other half.

- Heavier weights give more aspirational, higher-paying picks but lose accuracy.
- 0.5 and 0.3 sit at the balance point.

3.15 The four-year trend

Method. Fit the same per-skill model separately on the 2022, 2023, 2024 and 2025 surveys and compare each skill's percentage across years.

Results.

- **Steady:** Terraform and Kubernetes stay on top.
- **Falling:** Docker drops from +27% to +17%, and Redis and Go fall too. They are becoming mainstream, so knowing them sets people apart less.

Caveat. Each year has different respondents, so changes mix real market shifts with changes in who answered.

3.16 The honest limits (say these before you're asked)

1. **Association, not causation.** "Scala users earn 35% more than comparable developers" doesn't mean learning Scala adds 35%. Scala users may differ in employer or industry; only experience and country are controlled for.
2. **Survey data.** Respondents chose to take part, and pay is self-reported.
3. **USD at exchange rates**, not adjusted for cost of living.
4. **One year per model.**
5. **Judgement calls.** k = 300 peers, the 50-peer minimum and ±10 years were chosen by hand. Only the score weights were checked on held-out data.

3.17 The story in one paragraph (for your closing slide)

> *We measure each skill's value with regressions that compare developers within the same country and experience level, using log pay so effects are percentages. We find "people like you" with Jaccard-similarity nearest neighbours, and use what they know and want to learn as relevance and demand. We combine value, relevance and demand into a score, and value the final five-skill plan with one joint regression so overlapping skills aren't double-counted. Tested on held-out developers, the recommender finds people's real next skills 78% of the time, beating both a popularity baseline and a salary-only baseline.*

Part 4: Presentation plan (about 10–12 minutes)

Time	Section	What to say and show
0:00	**Problem**	Companies spend heavily on training, but people choose skills on instinct. Our question: which five skills should *this* developer learn next, with statistical evidence?
1:00	**Data**	2025 Stack Overflow survey: 19,210 working developers, 122 skills, 156 countries. Cleaning: USD pay (not local currency), an outlier rule on log pay, experience capped at 50.
2:00	**Method overview**	Three blocks: salary regressions with country fixed effects, KNN peers, and a joint regression for the plan. Show the cheat-sheet table.
3:00	**Live demo**	Follow the demo script below.
8:00	**Findings**	+3.7% per year; R^2 0.559; top and bottom skills; the AI Simpson's paradox (+9.9%); satisfaction is hard to predict; three personas.
10:00	**Validation**	78% hit rate against 72% and 5% baselines.
11:00	**Limitations and close**	Association, not causation; self-reported, self-selected sample; USD not purchasing-power adjusted; one year. Close with who the app helps: developers, L&D managers, HR analytics.

Demo script

1. **Open the app with the defaults (A1).** - **Say:** "With no stack, it shows the most valuable skills for a US full-stack developer: Docker, AWS, Terraform, Kubernetes and Snowflake." - **Point at:** the **+82%** headline and "5 of 5 significant".
2. **Add Python, JavaScript, SQL, PostgreSQL and Docker (B1).** - **Point at the constellation:** the amber-ringed picks sit beside it. - **Read the header:** 731 matching developers, of whom the 300 most similar are used. - **Picks:** TypeScript, VS Code, npm, React and AWS. The total is **+24%**. - **Explain one row:** "67% of my peers know TypeScript, 47% want it, and it's associated with +6.7% pay."
3. **Change the country to India (B5).** - **Say:** the median drops from $130,000 to **$15,000** and the picks change, because the model compares you within your country.
4. **Back to the US; move experience to 25 (B3).** - **Point at the movement tags:** AWS up 3, OpenAI GPT new. The median rises to **$160,000**.
5. **Scroll to the evidence table.** - **Explain:** stand-alone effect, interval, p-value, and the know and want columns.
6. **Scroll to the persona section.** - **Explain:** it's K-means on the stack only, so the country doesn't change it.
7. **Calculator: Kubernetes, Terraform, Snowflake.** - **Say:** this gives **+74%**, while separately they add to +92%. "Overlapping skills share one premium; the joint model doesn't double-count."
8. **Optional: set country to Nepal (B7).** - **Point at the relaxation note:** too few matches, so the app matches on role only and says so.

Part 5: Likely questions and answers

Q: Does learning Scala make you earn 35% more? No. It's an association: Scala users earn 35% more than comparable developers (same experience and country), but they may differ in employer, industry or seniority. Country and experience are controlled for; everything else is not.

Q: Why log pay? Pay is skewed. On a log scale the coefficients are percentages, and percentages are comparable across countries.

Q: Why country fixed effects? Without them, a skill used mostly in rich countries looks valuable just because of where its users live. An earlier version of the project made this mistake, and fixing it changed the sign of 12 skill estimates.

Q: Why is the plan total smaller than the sum of the five effects? Skills come in bundles and share one premium. One joint regression separates their effects, and its interval accounts for how the estimates move together (covariance).

Q: Why recommend VS Code if its joint pay effect is about zero? The score balances relevance and demand with pay. VS Code is used by 68% of your peers and wanted by 44%. The page openly shows its small pay contribution.

Q: What does "3 of 5 significant" mean? In the joint regression, three of the five skills have $p < 0.05$. Significant means clearly different from zero (in either direction), not large.

Q: Why is the premium negative? Your particular peer group earns slightly less than the average developer with the same experience in the same country. It's context only; it can't change the ranking because it's identical for every candidate.

Q: Why doesn't the persona change when I change the country? Personas are clusters of skills only.

Q: Why does a small .NET stack show "Lean generalist"? The .NET cluster's members list 21 skills on average. A 4-skill stack is closest to the small-stack cluster. Six .NET skills (adding ASP.NET and NuGet) flip it to ".NET stack".

Q: The silhouette is only 0.085. Are the clusters real? Low silhouettes are normal for sparse yes/no data with many dimensions. The clusters are clearly interpretable, and factor analysis independently finds the same ecosystems.

Q: Why not just recommend the highest-paying skills? We tested it: that finds a developer's real next skill only 5% of the time, against 78% for our model. The picks have to fit the person.

Q: How did you choose the weights 0.5 and 0.3? They were checked on held-out data. Heavier weights give more aspirational picks but lose accuracy; 0.5 and 0.3 sit on the balance between accuracy and value.

Q: AUC 0.57 is low. Is the classifier useless? It shows that satisfaction is mostly driven by things the survey doesn't measure. Three different models agree, so that is a finding.

Q: Is the AI result reliable? The controlled estimate is +9.9% (95% CI +7.3% to +12.7%, $p \approx 5\times10^{-14}$). It's still an association.

Q: What would you do next? Purchasing-power-adjusted pay, robust standard errors, country-specific skill effects, multi-year panel data, and learning the weights directly.

Part 6: Test scenarios with expected results

Reload the page to reset to the defaults before each scenario: Developer Full Stack, 5 years, United States of America, AI box ticked, empty stack. The headline and its interval are rounded to whole percents on the page.

A. Empty stack (cold start)

A1. Defaults

- **Group:** 731 developers. Median $130,000, premium −2.1%, persona "add your stack".
- **Picks:** Docker +16.4% (63% know it), AWS +21.4% (48%), Terraform +31.0% (18%), Kubernetes +28.0% (24%), Snowflake +33.0% (6%).
- **Headline: +82%** (+72% → +92%), 5 of 5 significant. The "want" column shows "—".

A2. A1, untick AI

- **Group:** 224 developers. Median $125,000, premium −11.3%.
- **Picks:** Kubernetes, Docker, Terraform, AWS, Snowflake.
- **Tags:** Kubernetes up 3, Docker down 1, AWS down 2.
- **Headline: +82%**, unchanged because it's the same five skills.

B. Base stack: Python, JavaScript, SQL, PostgreSQL, Docker

B1. Defaults plus the base stack (the report's worked example)

- **Header:** "Your 300 nearest peers… drawn from 731 developers".
- **Facts:** median $130,000, premium −2.0%, persona Lean generalist stacks.

#	Skill	Effect	Know	Want
1	TypeScript	+6.7%	67%	47%
2	VS Code	+3.2%	68%	44%
3	npm	+4.2%	63%	26%
4	React	+5.1%	56%	32%
5	AWS	+21.4%	51%	23%

- **Headline: +24%** (+20% → +28%), 3 of 5 significant.

B2. Experience 0

- **Group:** 520 developers (0–10 years). Median $119,500, premium −6.0%.
- **Picks:** VS Code, TypeScript, npm, React, AWS.
- **Headline: +24%** (+20% → +28%): the same five skills in a different order, so the same total.

B3. Experience 25

- **Group:** 490 developers (15–35 years). Median **$160,000**, premium −7.3%.

- **Picks:** VS Code, AWS, TypeScript, npm, OpenAI GPT (chat).
- **Tags:** VS Code up 1, AWS up 3, TypeScript down 2, npm down 1, OpenAI new.
- **Headline:** **+28%** (+24% → +32%), 4 of 5.

B4. Experience 50

- **Relaxation note:** peers matched on role and country (1,580 developers); experience and AI use not used.
- **Facts:** median $138,000, premium −9.8%.
- **Picks:** AWS, VS Code, TypeScript, npm, Bash/Shell.
- **Headline:** **+29%** (+25% → +33%), 4 of 5.

B5. Country India

- **Group:** 277 developers, so "Your 277 nearest peers". Median **$15,000**, premium −14.9%.
- **Picks:** VS Code, TypeScript, React, npm, OpenAI GPT (chat).
- **Headline:** **+14%** (+11% → +18%), 2 of 5.

B6. Country Germany

- **Group:** 341 developers. Median $74,249, premium −2.1%.
- **Picks:** TypeScript, npm, VS Code, OpenAI GPT (chat), Bash/Shell.
- **Headline:** **+20%** (+16% → +25%), 3 of 5.

B7. Country Nepal

- **Relaxation note:** peers matched on role only (6,984 developers); country, experience and AI use not used.
- **Facts:** median $80,605, premium −10.5%.
- **Picks:** VS Code, TypeScript, Bash/Shell, AWS, npm.
- **Headline:** **+29%** (+25% → +33%), 4 of 5.

C. Other roles and stacks

C1. Role Data Scientist; stack Python, SQL

- **Relaxation note:** peers matched on role and country (55 developers).
- **Facts:** median $150,000, premium **+14.5%**.
- **Picks:** VS Code, JupyterLab, PostgreSQL, Pip, Docker.
- **Headline:** **+21%** (+16% → +26%), 2 of 5.

C2. PHP stack: PHP, JavaScript, Laravel, MySQL

- **Facts:** median $115,340, premium −12.8%.
- **Picks:** VS Code, TypeScript, Docker, npm, React.
- **Headline:** **+18%** (+14% → +21%), 3 of 5.
- **Check:** no negative-effect skill is ever recommended.

C3. India; stack Python, JavaScript, SQL, PostgreSQL (no Docker)

- **Group:** 277 developers. Median $15,000, premium −16.0%.
- **Picks:** VS Code, TypeScript, React, Docker, npm.
- **Headline:** **+18%** (+14% → +21%), 3 of 5.

D. Personas

D1. C#, Microsoft SQL Server, Visual Studio, ASP.NET Core

- **Persona: Lean generalist stacks**. Median $117,000, premium −14.8%.
- **Picks:** VS Code, TypeScript, npm, Docker, OpenAI GPT (chat).
- **Headline:** **+21%** (+17% → +25%), 4 of 5.

D2. D1 plus ASP.NET and NuGet

- **Persona: .NET stack**. Median $117,000, premium −14.1%.
- **Picks:** the same five. **Headline:** **+21%**.

D3. TypeScript, React, Node.js, Vite, pnpm

- **Persona: Lean generalist stacks**. Median $133,650, premium +4.0%.
- **Picks:** VS Code, npm, Docker, PostgreSQL, AWS.
- **Headline:** **+28%** (+24% → +32%), 4 of 5.

D4. Full JS stack (13 skills)

- **Stack:**
- Languages: TypeScript, JavaScript, HTML/CSS
- Web frameworks: React, Node.js, Express, Next.js
- Cloud platforms: Vite, pnpm, npm, Yarn, Webpack
- Databases: PostgreSQL
- **Persona: Modern JS / web tooling**. Median $135,000, premium +1.4%.
- **Picks:** VS Code, Docker, AWS, OpenAI GPT (chat), Python.
- **Headline:** **+33%** (+29% → +38%), 4 of 5.

E. Calculator

#	Stack	Calculator picks	Expected	Interval	Joint effects (stand-alone)
E1	B1	Scala	**+35.1%**	+27.0% → +43.8%	Scala +35.1% (+35.1%)
E2	B1	Kubernetes	**+28.0%**	+25.2% → +30.9%	Kubernetes +28.0% (+28.0%)
E3	B1	Kubernetes, Terraform, Snowflake	**+73.7%**	+64.2% → +83.8%	Snowflake +20.8% (+33.0), Kubernetes +20.2% (+28.0), Terraform +19.6% (+31.0)
E4	C3	Docker, Kubernetes, AWS, Terraform	**+56.7%**	+51.7% → +61.9%	Kubernetes +17.4% (+28.0), Terraform +16.0% (+31.0), AWS +12.3% (+21.4), Docker +2.5% (+16.4)
E5	B1	PHP, Microsoft Access	**−38.7%**	−42.7% → −34.4%	PHP −19.1% (−19.8), Access −24.2% (−27.0), red markers

F. Checks that hold in every scenario

1. Every input change changes something visible: group size, median, premium, picks, order or tags.
2. A skill already in your stack is never recommended.
3. The sky chart shows your stack as joined white stars and the picks with amber rings.
4. The evidence table matches the plan list.
5. The ROI landscape stays the same whatever you enter.
6. Each skill box accepts at most 6 skills, and the calculator at most 8.